Méthode de calcul numérique

Limites de la machine

Rapport de projet

12/02/2024

BORDEAUX INP Enseirb-Matmeca

Table des matières

1 Partie 1 **2**
1.1 Objectif de la partie . . . 2
1.2 Créer la représentation et évaluer les erreurs relatives . . . 2
1.2.1 Tests de la représentation décimale et comparaison d'erreur relative selon la précision 3

2 Partie 2 **4**
2.1 Algorithmes de CORDIC . . . 4
2.1.1 Précision et approximation de calcul . . . 4
2.1.2 Cas de la calculatrice . . . 4
2.1.3 Calcul des fonctions usuelles . . . 4

Résumé

Le but principal de ce premier projet d'algorithmes numériques est d'appréhender la représentation des nombres en machines avec la partie 1. La deuxième partie a pour objectif quant à elle de comparer des erreurs avec des applications sur des fonctions qui sont l'algorithme népérien d'une part et ceux de CORDIC de l'autre.

Partie 1 – Partie 1

1.1 – Objectif de la partie

Comme énoncé dans le résumé introductif de la page précédente, cette partie va se concentrer sur la représentation de nombres décimaux réduits ainsi que leur utilisation pour pouvoir comparer des erreurs relatives lors de la manipulation de ces nombres tronqués.

1.2 – Créer la représentation et évaluer les erreurs relatives

Représentation décimale réduite

Étapes prises durant la création de l'algorithme :

- conversion du nombre en une chaine de caractères str_x ;
- stockage du signe dans s ;
- stockage du nombre de zéros après la virgule dans num_zeros ;
- stockage de la partie significative en new_x selon l'algorithme suivant :

```
new_x <- 0
pour i allant de db_partie_significative a fn_partie_significative
debut
    new_x <- new_x + str_x[i] x 10
fin
```

- arrondi au supérieur de la partie significative ;
- rajout des zéros de l'après partie significative à num_zeros si la virgule n'a pas encore été touché ;
- récupération du résultat final en calculant : $s \times new_x \times 10^{num_zeros}$

Erreur relative : la somme et le produit

Pour cette fonction représentant δ_s, nous avons utilisé la fonction précédente pour créer x et y en "machine" (c'est à dire leur représentation réduite) ainsi que leur somme en "machine".

Cette étape préalable permet que la somme de x et y "machine" simule bien la représentation de la somme dans une machine volontairement peu précise. Une fois ceci fait il suffit de calculer le delta avec le x et y donnés à la fonction.

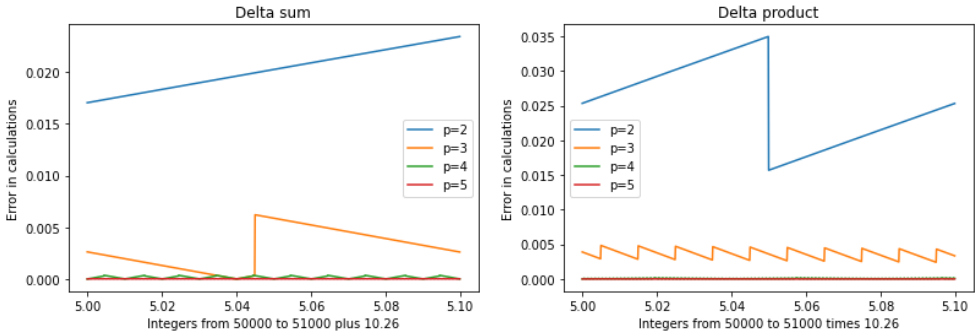
```
def delta_somme(x,y,p):
    x_machine = rp(x,p)
    y_machine = rp(y,p)
    return abs(x+y - rp((x_machine + y_machine),p))/abs(x+y)
```

De la même manière pour la multiplication , il suffit de séparer les calculs en "machine" et les calculs supposés réels avec une fonction identique à ci dessus mais en remplaçant la somme par le produit de compléxité similaire à la précédente.

1.2.1 – Tests de la représentation décimale et comparaison d'erreur relative selon la précision

Pour cette partie nous avons tester sur plusieurs valeurs la fonction rp dont celles présentées dans le fascicule de TD.

Nombre réel	Sur 4 décimales	Sur 6 décimales
3.141592658	3.142	3.1415900000000003
10507.1823	10510	10507.2
0.0001857563	0.0001858	0.000185756



(a) Addition (b) Multiplication

FIGURE 1 – Comparaison d'erreurs machines dans les opérations binaires

Les valeurs étudiées mettent en évidence des cycles dans les marges d'erreurs de la machine pour différentes précisions, avec des courbes régulières en dent de scie. En prenant en compte le fait que la figure 1a présente les courbes de valeurs sur un interval plus petit que la figure 1b, on retrouve des cycles similaires pour une même précision entre l'addition et la multiplication. De plus, plus la précision p est grande, plus la marge d'erreur reste proche de 0.

Ces cycles sont explicables par l'arrondissement des valeurs pour respecter les limites de la précision : pour une précision de 3, les valeurs 1.185 et 1.194 seront toutes les deux lues comme 1.19. Ces erreurs d'imprécision s'accumulent jusqu'à atteindre la valeur arrondie suivante ; une précision p plus grande permet d'atteindre cette valeur plus rapidement, et donc d'avoir une plus petite marge d'erreur.

Partie 2 – Partie 2

2.1 – Algorithmes de CORDIC

2.1.1 – Précision et approximation de calcul

En dehors des erreurs dues à la représentation en mémoire des valeurs, une imprécision dans les calculs peut être un effet secondaire d'une optimisation du reste du programme. Nous regardons ici le cas du calcul du logarithme népérien de 2 utilisant une approximation de la valeur :

$$log(2) = \sum_{n=0}^{\infty} \frac{(-1)^{n+1}}{n}$$

Une somme à l'infinie n'est pas possible dans une application concrète du calcul, le résultat ne pourra donc être qu'une approximation de ln(2) ; mais le calcul même ne demande pas une machine puissante.

2.1.2 – Cas de la calculatrice

Le cas de la calculatrice est un bon exemple de l'utilisation et de l'importance des algorithmes de CORDIC. En effet, les ressources étant limités (puissance de calcul, mémoire) il faut réussir à donner des résultats précis en limitant au maximum les erreurs d'approximations et dans un temps assez court pour satisfaire l'utilisateur.

La représentation des nombres dans une calculatrice est la suivante :

- Un nombre 'flottant' sur 8 octets
- Une mantisse constituée de 13 chiffres codés indépendant sur 4 chiffres binaires
- Un exposant
- Le signe du nombre (et de l'exposant s'il n'est pas signé)

Cette représentation des nombres possède des avantages et des défauts :

- Avantages :
  1. Rapidité d'exécution des calculs grace à une taille de nombre raisonable et une méthode de résolution par décalage de bits
  2. Une précision relative pour les réels satisfaisante
- Défauts
  1. Perte de précision pour les petits nombres, engendre des erreurs de plus en plus grande lors des calculs précis à cause de l'arrondissement des valeurs

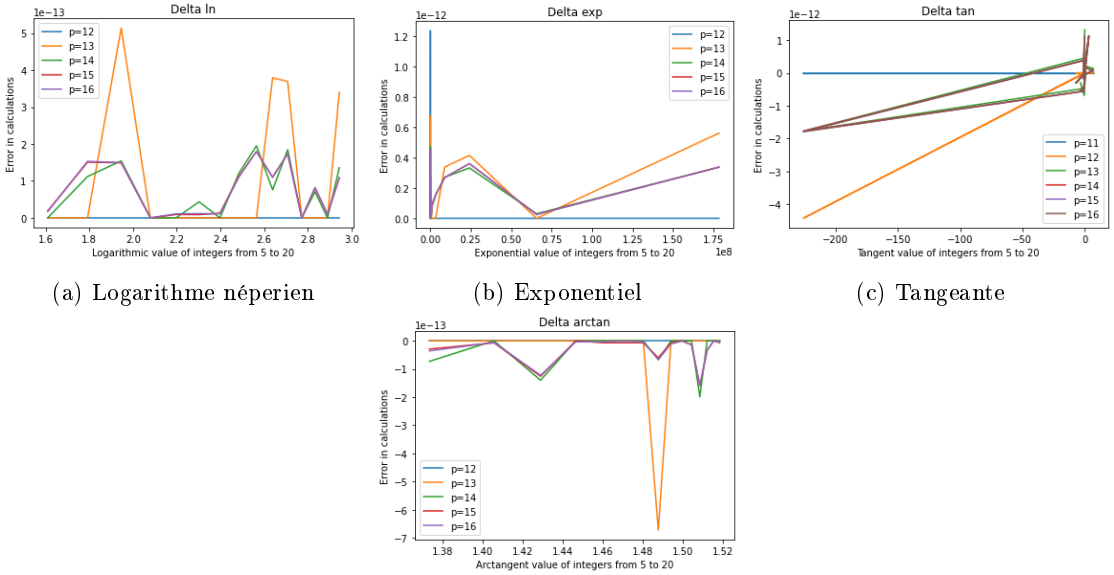
2.1.3 – Calcul des fonctions usuelles

La fin du projet s'intéresse à l'implémentation de 4 algorithmes pour calculer fonctions trigonométriques et exponentielle.

Ces 4 algorithmes reposent sur le principe d'éviter les multiplications et les divisions pour les calculs car ce sont des opérations lourdes sur une calculatrice. Afin d'éviter les opérations lourdes, les algorithmes fonctionnent de la manière suivante :

- **Réduction de la valeur d'entrée dans un intervalle approprié** : Pour les fonctions comme ln et exp, la valeur d'entrée est réduite à un intervalle où les méthodes de calcul sont efficaces. Par exemple, pour ln, la valeur est ramenée à l'intervalle [1, 10[.
- **Application de l'algorithme CORDIC** : Une fois que la valeur d'entrée est dans l'intervalle approprié, l'algorithme CORDIC est appliqué en itérant sur un ensemble précalculé de valeurs. Ces valeurs sont utilisées pour effectuer des transformations simples telles que l'addition, la soustraction et le décalage de bits (peu couteuse en machine). Chaque fonction suit des transformations différentes pour réduire l'entrée tout en calculant le résultat.
- **Interpolation linéaire pour précision supplémentaire** : Une fois les transformations appliquées, le résultat final est obtenu par interpolation linéaire ou par d'autres méthodes d'approximation pour obtenir une précision supplémentaire si nécessaire.

Les résultats obtenus sont corrects en deçà d'une certaine précision.



(a) Logarithme néperien (b) Exponentiel (c) Tangeante

(d) Arctangeante

FIGURE 1 – Comparaison d'erreurs machines dans les algorithmes CORDICs

Dans les applications des algorithmes CORDIC pour le logarithme, l'exponentielle, la tangeante et l'arctangeante, on observe des valeurs très aléatoires dans les marges d'erreurs qui ne sont pas corrélées à la valeur de la précision. Puisque ces calculs sont basés sur une approximation des valeurs recherchées, une plus petite précision dans les valeurs est préférable : ainsi on constate que les erreurs ne deviennent visibles qu'à partir d'une précision de douze chiffres.

Cette technique permet d'obtenir des résultats rapidement car la valeur d'entrée est réduite jusqu'à des valeurs ou les calculs peuvent être effectués efficacement.

Dans le contexte d'une calculatrice, cette technique est pertinente car elle évite les calculs lourds (respecte la limite de puissance de calcul de la calculatrice) De plus, elle effectue le calcul de manière rapide, pour fournir un résultat pertinent et rapidement à l'utilisateur en respectant les contraintes technologiques de la calculatrice (faible puissance de calcul et faible mémoire)